

1. What is a Dictionary?

A Python dictionary is analogous to a real-world dictionary. It stores data as a collection of **key-value pairs**, enclosed in curly braces {}.

- **Key:** A unique identifier for a piece of data. Keys must be of an *immutable* type (e.g., string, number, or tuple).
- **Value:** The data associated with a key. This can be *any* data type (e.g., string, number, list, or another dictionary).

Python dictionaries internally use a HashMap.

Characteristics of Dictionary

1. Ordered

- From Python 3.7+, dictionaries **preserve insertion order**.
(The order in which you insert items is the same order Python remembers.)

2. Mutable

- You can **change**, **add**, or **remove** items after the dictionary is created.

3. Key-Value Structure

- Data is stored as: { key : value }

4. Keys Must Be Unique

- Two items cannot have the same key.

5. Keys Must Be Immutable

Valid key types:

- String
- Number
- Tuple

Invalid:

- List
- Dictionary

6. Values Can Be Anything

- Strings, lists, dictionaries, objects, functions, etc.

Creating a Dictionary

A dictionary can be defined as follows:

```
# A dictionary representing a student
student = {
    "name": "Alex",
    "grade": 10,
    "subjects": ["Math", "History", "Art"],
    "is_passing": True
}
```

```
print(student)
```

Output: {'name': 'Alex', 'grade': 10, 'subjects': ['Math', 'History', 'Art'], 'is_passing': True}

```
# An empty dictionary can also be created
empty_dict = {}
```

2. Basic Operations

The following are the three most basic operations performed on a dictionary.

Accessing a Value

Use the key inside square brackets [] to retrieve its associated value.

```
print(student["name"])
```

Output: Alex

```
print(student["subjects"])
```

Output: ['Math', 'History', 'Art']

Note: If you attempt to access a key that does not exist, Python will raise a `KeyError`. A *safer* method is described in the "Methods" section.

Adding or Updating a Value

A new key-value pair can be added, or an existing value updated, using the same [] assignment syntax.

```
# Add a new key-value pair
student["school"] = "West High"
print(student)
```

Output: {... 'is_passing': True, 'school': 'West High'}

```
# Update the value for an existing key
student["grade"] = 11
print(student["grade"])
```

Output: 11

Deleting a Value

Use the `del` keyword to remove a key-value pair.

```
del student["is_passing"]
print(student)
```

Output: {'name': 'Alex', 'grade': 11, 'subjects': ['Math', 'History', 'Art'], 'school': 'West High'}

3. Common Dictionary Methods

Methods are functions that are part of the dictionary object. They are called using dot notation (`.`).

.get(key, default)

This is the **safe method** to access a value.

- If the key exists, it returns the value.
- If the key **does not exist**, it returns `None` (or the specified default value) instead of raising an error.

```
# 'name' exists
print(student.get("name"))
```

Output: Alex

```
# 'age' does not exist
print(student.get("age"))
```

Output: None

```
# 'age' does not exist, so the default value is returned
print(student.get("age", 16))
```

Output: 16

.keys(), .values(), and .items()

These methods are useful for iteration.

- `.keys()`: Returns a view object displaying all the keys.
- `.values()`: Returns a view object displaying all the values.
- `.items()`: Returns a view object displaying all the (key, value) pairs as tuples.

This is most useful for iterating through a dictionary:

```
# .items() is the most common method for iteration
for key, value in student.items():
    print(f"The key is '{key}' and the value is '{value}'")
```

Output:

```
# The key is 'name' and the value is 'Alex'
# The key is 'grade' and the value is '11'
# ...and so on
```

.update(other_dictionary)

This method merges the key-value pairs from one dictionary into another. If any keys are duplicated, the values from the *other_dictionary* will overwrite the existing ones.

```
student = {"name": "Alex", "grade": 10}
new_info = {"grade": 11, "hometown": "Boston"}
```

```
student.update(new_info)
print(student)
```

Output: {'name': 'Alex', 'grade': 11, 'hometown': 'Boston'}

```
# 'grade' was updated and 'hometown' was added.
```

.pop(key)

This method **removes** a key-value pair from the dictionary and **returns** the corresponding value. This is useful for retrieving a value and simultaneously deleting it.

```
student = {"name": "Alex", "grade": 10}
```

```
alex_grade = student.pop("grade")
print(alex_grade)
```

Output: 10

```
print(student)
```

Output: {'name': 'Alex'}

.popitem()

This method removes and returns the *last* key-value pair inserted into the dictionary (in Python 3.7+).

```
student = {"name": "Alex", "grade": 10, "school": "West High"}
```

```
last_item = student.popitem()  
print(last_item)
```

Output: ('school', 'West High')

```
print(student)
```

Output: {'name': 'Alex', 'grade': 10}

.copy()

This method returns a *shallow copy* of the dictionary. This is important when modifications are needed on a new dictionary without affecting the original.

```
original = {"a": 1, "b": 2}
```

```
# Create a copy  
copied = original.copy()
```

```
# Modify the copy  
copied["c"] = 3
```

```
print(original)
```

Output: {'a': 1, 'b': 2} (The original is unchanged)

```
print(copied)
```

Output: {'a': 1, 'b': 2, 'c': 3} (The copy reflects the change)

.clear()

This method removes *all* items from the dictionary, rendering it empty.

```
student = {"name": "Alex", "grade": 10}
```

```
student.clear()
print(student)
Output: {}
```

4. Other Useful Methods

The following methods are useful for specific situations.

.setdefault(key, default)

This method is an extension of `.get()`.

- If the key exists, it returns the value (identical to `.get()`).
- If the key **does not exist**, it **inserts** the key with the default value and then returns that default value.

This is useful for adding items only if they are not already present.

If you **do not pass the default parameter**, Python automatically uses **None** as the default value.

```
student = {"name": "Alex", "grade": 10}
```

```
# 'grade' exists, so its value is returned
student.setdefault("grade", 12)
print(student)
```

Output: {'name': 'Alex', 'grade': 10}

```
# 'club' does not exist, so it is added with the default value
student.setdefault("club", "Art Club")
print(student)
```

Output: {'name': 'Alex', 'grade': 10, 'club': 'Art Club'}

.fromkeys(iterable, value)

This is a static method used to create a *new* dictionary from an iterable (like a list) of keys. All keys in the new dictionary will be assigned the same value.

Note: This method is called on the `dict` class itself, not on an existing dictionary instance.

```
keys = ["name", "grade", "school"]
default_value = "unknown"
```

```
# Create a new dictionary from the list of keys
new_student = dict.fromkeys(keys, default_value)
print(new_student)
```

Output: {'name': 'unknown', 'grade': 'unknown', 'school': 'unknown'}

```
# If no value is provided, it defaults to None
new_student_none = dict.fromkeys(keys)
print(new_student_none)
```

Output: {'name': None, 'grade': None, 'school': None}

5. Dictionary Comprehension

A **comprehension** is a concise syntax for creating a new dictionary, often from an existing iterable or dictionary. It provides a shortcut for a for loop.

Example: Creation using a for loop

For example, to create a dictionary from a list of numbers, where keys are the numbers and values are their squares:

```
numbers = [1, 2, 3, 4, 5]
squares_dict = {} # Initialize an empty dictionary
```

```
for num in numbers:
    squares_dict[num] = num * num
```

```
print(squares_dict)
```

Output: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

Example: Creation using a Comprehension

The entire for loop can be written in one line using a dictionary comprehension:

```
numbers = [1, 2, 3, 4, 5]
```

```
# This is the dictionary comprehension
# {key: value for item in iterable}
squares_dict = {num: num * num for num in numbers}
```

```
print(squares_dict)
```

Output: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

1. Using zip() in Dictionary Comprehension

The zip() function takes two or more iterables (like lists or tuples) and aggregates them into a single iterator of tuples. This is the standard method for merging two separate lists into a single dictionary (Key-Value pairs).

```
products = ["Laptop", "Mouse", "Monitor"]  
  
prices = [1000, 25, 200]  
  
# Create a dictionary mapping products to prices  
  
price_catalog = {product: price for product, price in zip(products, prices)}  
  
print(price_catalog)
```

Output: {'Laptop': 1000, 'Mouse': 25, 'Monitor': 200}

2. Using enumerate() in Dictionary Comprehension

The enumerate() function adds a counter (index) to an iterable. This is useful when you need to use the position of an item as the dictionary key or as part of the value logic.

```
students = ["Alice", "Bob", "Charlie"]  
  
# Create a dictionary with ID numbers starting at 1  
  
student_db = {index + 1 : name for index, name in enumerate(students)}  
  
print(student_db)
```

Output: {1: 'Alice', 2: 'Bob', 3: 'Charlie'}

Comprehension with a Condition

An if statement can also be included to filter which items are added. To create a new dictionary that only includes students with a passing score:

```
student_scores = {"Alex": 92, "Beth": 78, "Charlie": 100, "David": 59}  
  
# {key: value for (key, value) in dict.items() if condition}  
passing_students = {name: score for (name, score) in student_scores.items() if score >= 60}
```



```
print(passing_students)
```

Output: {'Alex': 92, 'Beth': 78, 'Charlie': 100}